Learning Objectives

- Understand how a lexer turns a stream of characters into tokens.
- Design and extend the token interface of a compiler.
- Track source positions and preserve the textual representation of tokens.
- Recognize literals, identifiers, operators, and punctuation.
- Build the lexer that will be used throughout the remaining compiler project.

Keeping Your Work

Keep the code you develop for the compiler project in a `not-abc` directory. If you are already familiar with Git, you may instead create a `not-abc` repository and keep your work there. We will introduce Git in a later worksheet, so **no Git experience is required at this point**.

Starting Point

As our starting point, we use the lexer developed in the lecture videos (`lexer.hdr`, `lexer.abc`, and `xtest_lexer.abc`). You can find the source files on Moodle and GitHub.

For this worksheet, the code in `lexer.abc` has been slightly reorganized into an internal part and an exported part that can be used by other translation units:

`lexer.hdr`

```
enum TokenKind
{
    BAD_TOKEN,
    EOI,
    DECIMAL_LITERAL,
    PLUS,
};

extern token: TokenKind;

extern fn getToken(): TokenKind;
```

`lexer.abc`

```
@ <stdio.hdr>
@ "lexer.hdr"

// -- internal: only available within this translation unit

global ch: int;

// -- exported: available for other translation units
global token: TokenKind;

fn getToken(): TokenKind
{
    while (ch == 0 || ch == ' ' || ch == '\n' || ch == '\t') {
        ch = getchar();
    }
    if (ch >= '0' && ch <= '9') {
        while (ch >= '0' && ch <= '9') {
            ch = getchar();
        }
        return token = DECIMAL_LITERAL;
    } else if (ch == '+') {
        ch = getchar();
        return token = PLUS;
    } else if (ch == EOF) {
        return token = EOI;
    } else {
        ch = getchar();
        return token = BAD_TOKEN;
    }
}
```

You can build the test program `xtest_lexer` in the terminal with

```
abc xtest_lexer.abc lexer.abc
```

During this worksheet, we will add more source files and test programs. Since we are not using Makefiles yet, it is convenient to build a test program as follows:

```
abc xtest_lexer.abc [^x]*.abc
```

The shell expands `[^x]*.abc` to all `.abc` files in the current directory whose names do not start with `x`. By convention, we use the `x` prefix for our test programs.

© Michael C. Lehn | CC BY-SA 4.0

Step 1 — Simplifying the Test Program and Introducing Switch Statements

The test program should be simplified so that we no longer need a large number of `if` statements to map the value of token to a string. Instead, we will use a function `tokenKindStr` as follows:

```
@ <stdio.hdr>
@ "lexer.hdr"

fn main()
{
    while (getToken() != EOI) {
        printf("%s\n", tokenKindStr(token));
    }
}
```

The function `tokenKindStr` shall be implemented in `lexer.abc`. We will use this opportunity to introduce the `switch` statement. We will discuss the details shortly; for now, you can simply add the following code at the end of `lexer.abc`:

```
fn tokenKindStr(token: TokenKind): ->readonly char
{
    local str: -> char = "??";
    switch (token) {
        case BAD_TOKEN:
            str = "BAD_TOKEN";
            break;
        case EOI:
            str = "EOI";
            break;
        case DECIMAL_LITERAL:
            str = "DECIMAL_LITERAL";
            break;
        case PLUS:
            str = "PLUS";
            break;
    }
    return str;
}
```

Tasks

1. Adapt `lexer.hdr` so that `tokenKindStr` can be used from other translation units.
2. A quick review of familiar concepts:
   - Make sure you understand that the parameter `token` and the global variable `token` are two different variables. Recall the concept of a variable's scope.
   - `str` is a pointer to a character, whereas `"BAD_TOKEN"` is an array of characters (the compiler creates this array as a global object). Thus, in the assignment `str = "BAD_TOKEN"` an implicit conversion takes place: `str` is assigned the address of the first character of the array.
3. A `switch` statement transfers control to a `case` label depending on the value of an expression — in this example, the expression is token. The expression must have an integer type (no floating-point types, structs, etc.), and the values used in the `case` labels must be constant. Once a `case` label has been selected, execution continues from there until either the end of the switch block is reached or a break statement leaves the block.
   (a) **Fall-through in switch statements:** Remove the `break` statements from the `switch` block. Test the program with different inputs. Explain what you observe.
   (b) **The value range of enums:** Add the following statement to main in `xtest_lexer.abc`, before the while loop:

   ```
   printf("%s\n", tokenKindStr(42));
   ```

   No symbolic name was declared for the value `42` in `TokenKind`. Nevertheless, a variable of type TokenKind can hold this value.
   (c) **Exhaustiveness checking:** Remove any one of the `case` labels. The compiler should now complain that not all constants declared in `TokenKind` are covered by the `case` labels.

© Michael C. Lehn | CC BY-SA 4.0

Step 2 — The Default Label in Switch Statements

Change the function `tokenKindStr` as follows:

```
fn tokenKindStr(token: TokenKind): -> char
{
    switch (token) {
        case BAD_TOKEN: return "BAD_TOKEN";
        case EOI: return "EOI";
        case DECIMAL_LITERAL: return "DECIMAL_LITERAL";
        case PLUS: return "PLUS";
        default: return "??";
    }
}
```

The `switch` block now contains a `default` label. Control is transferred to this label whenever none of the `case` labels matches.

The function now also has multiple exit points, since each label directly returns from the function. Multiple exit points can make the control flow harder to follow. In this particular case, however, they have the advantage that accidental fall-through between cases is no longer possible.

Task

1. Verify that the `default` label does not have to appear last. For example, move it to the beginning of the `switch` block.
2. Remove one of the `case` labels. Verify that the compiler no longer complains that not all constants declared in `TokenKind` are covered by case labels.
3. Use whichever style you prefer for your implementation of `tokenKindStr`. And stick to it.

Step 3 — A New Token

The lexer shall now recognize the character `*` as a `token`. Use `ASTERISK` as the corresponding `enum` constant.

Make all necessary changes. This includes making sure that you can test whether the new token is recognized correctly.

Step 4 — Row and Column Position of a Token (Preparation)

For each token found by the lexer, we also want to keep track of the row and column at which it was found. In this step, we first adapt the interface; in the next step, we will implement the new feature.

In `lexer.hdr`, declare a struct type `Token` with a member `kind` for the kind of token and a member for its position. The type of the global variable `token` is changed to `Token`:

```
struct Token
{
    kind:   TokenKind;
    pos:    struct Pos {
                row, col: unsigned;
            };
};

extern token: Token;
```

Several places in our project now have to be adapted. For now, make only the following changes:

1. In `xtest_lexer.abc`, we want to test the new feature. Replace the `printf` statement with

   ```
   printf("%u.%u: %s\n", token.pos.row, token.pos.col,
          tokenKindStr(token.kind));
   ```

   The line break is necessary because a programmer's brain can only process lines of at most 80 characters.
2. In `lexer.abc`, change the type of `token` to `Token` in its definition.
3. In `lexer.abc`, adapt each `return` statement so that it sets `token.kind`. For example, after finding a `+`, the `return` statement becomes

   ```
   return token.kind = PLUS;
   ```

© Michael C. Lehn | CC BY-SA 4.0

Step 4 is complete when the input

```
123 +
```

produces output like this:

```
123 +
0.0: DECIMAL_LITERAL
0.0: PLUS
```

Step 5 — Row and Column Position of a Token (Implementation)

Now that we can test the new feature, it makes sense to start implementing it. The idea is to call our own function `nextCh` instead of `getchar`. This function does two things:
1. Read a character with `getchar` and store it in `ch`.
2. Update the current position stored in the global variable `currentPos`.

Here is a first implementation of nextCh in `lexer.abc` (tabs are not handled correctly yet):

```
// -- internal: only available within this translation unit

global ch: int;
global currentPos: Pos = {1, 0};

fn nextCh()
{
    ch = getchar();
    if (ch == '\n') {
        ++currentPos.row;
        currentPos.col = 0;
    } else {
        ++currentPos.col;
    }
}
```

We can now use `nextCh` in `getToken` by making the following changes:

- After skipping all whitespace, set `token.pos` to `currentPos`:

```
    while (ch == 0 || ch == ' ' || ch == '\n' || ch == '\t') {
        nextCh();
    }
    token.pos = currentPos;
```

- Replace all statements of the form

  ```
  ch = getchar();
  ```

  with

  ```
  nextCh();
  ```

Implement these changes and test your lexer. Here is the output of one possible test:

```
MCL:hpc0-abc lehn$ ./a.out
123 + *
1.1: DECIMAL_LITERAL
1.5: PLUS
1.7: ASTERISK
1 2 3
2.1: DECIMAL_LITERAL
2.3: DECIMAL_LITERAL
2.5: DECIMAL_LITERAL
```

© Michael C. Lehn | CC BY-SA 4.0

Step 6 — Keeping Track of the Characters of a Token (Preparation)

Currently, our lexer can recognize, for example, that the input contains a decimal literal. However, the characters that make up this literal are not stored in token. We will change this now.
In `lexer.hdr`, define the type `TokenVal` for a string of up to 12 characters[1]. We will take care of strings of arbitrary length later. Then add a member `val` of this type to `Token`:

```
type TokenVal: array[13] of char;

struct Token
{
    kind:   TokenKind;
    pos:    struct Pos {
                row, col: unsigned;
            };
    val:    TokenVal;
};
```

In `xtest_lexer.abc`, replace the `printf` statement with

```
        printf("%u.%u: %s (%s) \n", token.pos.row, token.pos.col,
               tokenKindStr(token.kind), token.val);
```

Everything should already compile and run again. Verify that `token.val` is always an empty string. Is this defined or undefined behavior? Explain.

Step 7 — Keeping Track of the Characters of a Token (Implementation)

The following code from `lexer.abc` shows a first hack for quickly and easily supporting this feature:

```
// -- internal: only available within this translation unit

global ch: int;
global currentPos: Pos = {1, 0};
global updateVal: bool;
global lengthVal: size_t;

fn nextCh()
{
    if (updateVal) {
        token.val[lengthVal++] = ch;
        token.val[lengthVal] = 0;
    }
    ch = getchar();
    if (ch == '\n') {
        ++currentPos.row;
        currentPos.col = 0;
    } else {
        ++currentPos.col;
    }
}
```

There are two new global variables:
- `updateVal` determines whether the current character is appended to the value of the token.
- `lengthVal` keeps track of how many characters have already been appended to `token.val`. The type `size_t` is an unsigned integer type whose actual type is chosen by the compiler depending on the platform. It is guaranteed to be able to represent the size of any object — and therefore, in particular, the size of an array.

---

[1] For once, the array dimension is not 42. We deliberately choose a rather small value because this limitation is supposed to hurt eventually — and make it obvious that we need a better solution. Besides, $1 + 3 = 4$ and $3 - 1 = 2$, so 42 is still in there. ;-)

© Michael C. Lehn | CC BY-SA 4.0

We can now make the following change in `getToken`:

```
updateVal = false;
lengthVal = 0;
while (ch == 0 || ch == ' ' || ch == '\n' || ch == '\t') {
    nextCh();
}
token.pos = currentPos;
updateVal = true;
```

When entering the function, `lengthVal` and `updateVal` are reset. We then skip all whitespace characters. After that, all further characters are appended to `token.val`.

Tasks

1. The code does not check whether there is actually enough space left in `token.val` for another character. Test what happens when a token contains too many characters.
2. The following assertion is supposed to prevent undefined behavior when a token contains too many characters:

   ```
   if (updateVal) {
       assert(lengthVal < sizeof(token.val) / sizeof(token.val[0]));
       token.val[lengthVal++] = ch;
       token.val[lengthVal] = 0;
   }
   ```

   If the condition passed to `assert` is false, program execution is terminated with an assertion failure.

   Test whether this really fixes the problem. If not, correct the code.

   Hint: Of course it doesn't!!11oneeleven

Step 8 — Tidying Up the Code

Add the following function to `lexer.abc` and use it in the implementation of `getToken`. A good place for this function is near the beginning of the file, after the global variables and before the functions that use it.

```
fn isDigit(ch: int): bool
{
    return ch >= '0' && ch <= '9';
}
```

Step 9 — Punctuators

Our lexer still recognizes only a few punctuators. Extend it so that it recognizes all of the following:

Character Stream	TokenKind	Character Stream	TokenKind	Character Stream	TokenKind
=	EQUAL	|	VBAR	;	SEMICOLON
==	EQUAL2	||	VBAR2	{	LBRACE
<	LESS	!	NOT	}	RBRACE
<=	LESS_EQUAL	!=	NOT_EQUAL	+	PLUS
>	GREATER	#	GATTER	-	MINUS
>=	GREATER_EQUAL	(	LPAREN	*	ASTERISK
&	AMPERSAND	)	RPAREN	/	SLASH
&&	AMPERSAND2	,	COMMA	%	PERCENT

`GATTER` is a rather old-school German name for the `#` character. Feel free to use `HASH` instead. ;-)

© Michael C. Lehn | CC BY-SA 4.0

Some of these punctuators consist of **two characters**. This introduces a small complication: after reading the first character, the lexer does not always know yet which token it has found.
For example, `<` can either form the token `LESS` or be the first character of `LESS_EQUAL`. We can handle this in `getToken` as follows:

```
// ...
} else if (ch == '<') {
    nextCh();
    if (ch == '=') {
        nextCh();
        return token.kind = LESS_EQUAL;
    }
    return token.kind = LESS;
// ...
```

Make the corresponding changes to `TokenKind` and `tokenKindStr`, and extend `getToken` to recognize all punctuators listed above. Make sure that your test program allows you to verify that both one-character and two-character punctuators are recognized correctly.

Yes, this is somewhat tedious work. Looking at the table above, one might almost get the idea of writing a program that reads such a table and generates this code for us. ;-)
Good Vim skills can also make this kind of repetitive work considerably less tedious. If you are not convinced yet that learning Vim is worthwhile, here is another opportunity to reconsider. ;-)

At the very least, test all punctuators together. For example, the input

```
= == < <= > >= & && | || ! != # ( ) , ; { } + - * / %
```

should produce:

```
1.1: EQUAL (=)
1.3: EQUAL2 (==)
1.6: LESS (<)
1.8: LESS_EQUAL (<=)
1.11: GREATER (>)
1.13: GREATER_EQUAL (>=)
1.16: AMPERSAND2 (&)
1.18: AMPERSAND (&&)
1.21: VBAR (|)
1.23: VABR2 (||)
1.26: NOT (!)
1.28: NOT_EQUAL (!=)
1.31: GATTER (#)
1.33: LPAREN (()
1.35: RPAREN ())
1.37: COMMA (,)
1.39: SEMICOLON (;)
1.41: LBRACE ({)
1.43: RBRACE (})
1.45: PLUS (+)
1.47: MINUS (-)
1.49: ASTERISK (*)
1.51: SLASH (/)
1.53: PERCENT (%)
```

© Michael C. Lehn | CC BY-SA 4.0

Step 10 — Recognizing Identifiers

An identifier starts with a letter (`'a'` to `'z'` or `'A'` to `'Z'`) or an underscore (`'_'`). It may be followed by any number of letters, digits (`'0'` to `'9'`), or underscores.
Extend the lexer so that it recognizes identifiers. Implement and use a function `isLetter`, and add the enum constant `IDENTIFIER` to `TokenKind`.
Adapt the remaining code accordingly. In particular, extend `tokenKindStr` and the test program so that you can verify that identifiers are recognized correctly.
For example, the input `k1 k_1 abc_ABC123` should produce:

```
1.1: IDENTIFIER (k1)
1.4: IDENTIFIER (k_1)
1.8: IDENTIFIER (abc_ABC123)
```

Step 11 — Recognizing String and Character Literals

The lexer shall now also recognize **string literals** and **character literals**.
A string literal starts and ends with a double quote (`"`). Between these two double quotes, any character except `"` is allowed for now. Escaping with a backslash, for example `\"`, does not have to be supported yet. If you like, you may of course already implement it. Add the enum constant `STRING_LITERAL` to `TokenKind`.
A character literal starts and ends with a single quote (`'`). Between these two single quotes, exactly one character other than `'` is allowed for now. Again, escaping with a backslash, for example `'\n'`, does not have to be supported yet, but you are welcome to implement it. Add the enum constant `CHARACTER_LITERAL` to `TokenKind`.
It is important that you test the new feature. In particular, adapt `tokenKindStr` and the test program so that you can verify that string and character literals are recognized correctly and printed together with their values. For example, the input `'a' '\n' "hello\n"` should produce:

```
1.1: CHARACTER_LITERAL ('a')
1.5: CHARACTER_LITERAL ('\n')
1.10: STRING_LITERAL ("hello\n")
```

Note: The opening and closing quotes are part of the token's value (`token.val`). This applies to both string and character literals.

Chillout — Single-Line Comment

Our lexer is almost done. As a final small feature, let us add support for **single-line comments**.
A single-line comment starts with `//` and extends to the end of the current line. We already recognize `/` as the token `SLASH`, so only a small change to `getToken` is needed:

```
// ...
} else if (ch == '/') {
    nextCh();
    if (ch == '/') {
        updateVal = false;
        while (ch != '\n' && ch != EOF) {
            nextCh();
        }
        return getToken();
    }
    return token.kind = SLASH;
// ...
```

Make this change and test it with a few inputs containing single-line comments.
Take a moment to understand the return `getToken();` statement. After skipping the comment, `getToken` simply calls itself to find the next token. So, we get to meet recursion again. :-)
For example, the input

```
// comment
a = 42; // another comment
a = a + 1;
```

should produce:

```
2.1: IDENTIFIER (a)
2.3: EQUAL (=)
2.5: DECIMAL_LITERAL (42)
2.7: SEMICOLON (;)
3.1: IDENTIFIER (a)
3.3: EQUAL (=)
3.5: IDENTIFIER (a)
3.7: PLUS (+)
3.9: DECIMAL_LITERAL (1)
3.10: SEMICOLON (;)
```

© Michael C. Lehn | CC BY-SA 4.0